Applying Reinforcement Learning Methods to Physical Systems

by

Jacob Kruse

(Under the Direction of Prashant Doshi)

Abstract

This section of the page is where your abstract will go. Fill in this spot in the dissertation.tex file with your abstract. Doctoral 350 words max, 150 max for masters.

Index words: Model-Based Reinforcement Learning, Offline Reinforcement Learning, Robotic Grasping, Robotic Manipulators, Regularization, Ensemble Networks

Applying Reinforcement Learning Methods to Physical Systems

by

Jacob Kruse

Bachelor of Science, Western Illinois University, 2024

A Thesis Submitted to the Graduate Faculty of the

University of Georgia in Partial Fulfillment of the Requirements for the Degree.

Master of Science

Athens, Georgia

2026

©2026

Jacob Kruse

All Rights Reserved

Applying Reinforcement Learning Methods to Physical Systems

by

Jacob Kruse

Major Professor: Prashant Doshi

Committee: Jin Lu

Mehmet Kurum

Electronic Version Approved:

Ron Walcott

Dean of the Graduate School

The University of Georgia

Month Year

Dedication

Acknowledgments

Contents

Dedication **iv**

Acknowledgments **v**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**

1.1 Contributions 2

1.2 Thesis Structure 2

2 Related Work **3**

2.1 Learning Success Models for Grasping 3

2.2 Offline Reinforcement Learning 4

2.3 Model-Based Reinforcement Learning 4

3 Preliminaries **6**

3.1 Contextual Bandit Framework 6

3.2 Reward Function Modeling 7

4 Methodolgy **9**

4.1 Approximating and Accurate Reward Model 9

4.2 Iterative Policy Refinement 11

5 Experiments **15**

5.1 Task Description 15

5.2 Context/State Estimation 16

5.3 Context/State Definition 17

5.4 Action Definition 18

5.5 Reward Definition 19

5.6 Initial Dataset 20

6 Conclusion **21**

Appendices **22**

A **22**

Bibliography **23**

List of Figures

List of Tables

Chapter 1

Introduction

Reinforcement Learning (RL) has achieved remarkable success in simulation, where agents can execute many actions and easily evaluate rewards. For instance, Vecerik et al., 2017 trains a robotic manipulator in simulation to perform a variety of manipulation tasks, while Mnih et al., 2015 applies RL to teach an agent to play Atarti games. Recently, RL has been increasingly applied to real-world tasks, motivated by the desire to tackle challenging problems beyond simulation. For example, Smith et al., 2022 demonstrates how a legged robot can learn to traverse real world terrains, and Wu et al., 2023 shows the application of their method to multiple physical platforms, including a quadruped robot, a robotic manipulator, and Sphero robot. Another prominent application of real-world RL is robotic grasping Han et al., 2023, due to its widespread relevance in manipulation tasks. However, practical deployment of RL in the real world faces several challenges. Uncertainty in sensing and actuation, high-dimensional continuous state and action spaces, the high cost of collecting large amounts of data, and the need to ensure safety during exploration all complicate the learning process.

Offline RL is a popular strategy for physical RL because it allows agents to learn from pre-colelcted datasets. This greatly improves the sample efficiency by enabling the algorithm to focus entirely on policy optimization rather than exploration. To address the main challenge of distribution shift in offline RL, techniques such as regularization and policy constraints are commonly applied. Regularization involves adding a term to the learning objective that encourages desirable behaviors in the learned policy, such as

safer actions, improved exploration, or policy outputs that stay close to the data distribution. Similarly, Maximum Entropy Reinforcement Learning (MaxEnt RL) Haarnoja et al., 2017 alters the standard RL objective by adding an entropy term to the reward. This encourages the policy to maximize both expected return and entropy, promting stochasticity in the policy, Although this resembles regularization with the additional term, MaxEnt RL fundamentally redefines the optimization objective. A notable example is Soft Actor-Critic (SAC) Haarnoja et al., 2018, which optimizes the MaxEnt objective. The method proposed in this work takes a similar approach by redefining the objective to include an additional term aimed at increasing action robustness.

1.1 Contributions

In this paper, we introduce a framework that learns an approximate reward model and leverages it for iterative policy optimization with a novel robustness-oriented objective. Our approach is inspired by ideas from SAC and MaxEnt RL but adapts them to a contextual bandit setting suitable for grasping tasks. The main contributions of this work are:

- A new learning objective, the Maximum Action Noise Threshold (MaxANT), which encourages selection of actions that are both high-reward and robust to perturbations.
- Experiments demonstrating the method on a robotic sorting task, showing improved performance and robustness compared to standard policy optimization.
- An iterative learning algorithm that refines both the reward model and policy by combining expert, random, and on-policy data to efficiently improve grasping performance.

1.2 Thesis Structure

Chapter 2

Related Work

2.1 Learning Success Models for Grasping

Robotic grasping is a major area of study within robotics due to its wide use in manipulation tasks. A significant portion of research focused on robotic grasping has employed the use of learning methods, several of which include the learning of a grasp success model. These models are used to predict the success of certain gripping actions with state information from RGB images, object positions, or affordance representations. After collecting data from 50,000 grasp attempts, Pinto and Gupta, 2016 trained a Convolutional Neural Network (CNN) to take image patches as input and output the probability of grip success for candidate grasp angles. Similarly, Levine et al., 2018 learned a grasp prediction network that predicts the likelihood of a successful grasp and integrated this estimate in a continuous servo controller. After generating a dataset of synthetic point clouds, Mahler et al., 2017 learned a grip robustness function that scores candidate grasps and enables planning by selecting the highest rated action. Other methods, such as Asif et al., 2019 and Detry et al., 2011, learn these grasp success models and utilize them for simple evaluation or to find high success actions. The method in this paper follows this line of work by learning a grasp success model that maps states and actions to success probabilities. The learned success predictor in this work is interpreted as a reward function, and utilized to learn an effective grasping policy.

2.2 Offline Reinforcement Learning

Offline Reinforcement Learning Prudencio et al., 2023 learns a policy $\pi_{\mathcal{D}}$ from a fixed dataset collected by an exploration policy π_{Explore} without further environment interaction. Unlike other learning methods that employ the use of training datasets, the goal of Offline RL is the same as standard Online RL, which is to learn a policy that maximizes expected reward. Because training is performed on static, batched data, Offline RL can be more sample-efficient and optimize faster than Online RL. However, a central challenge with Offline RL is distribution shift: the learned policy $\pi_{\mathcal{D}}$ may end up in states or take actions during training that are not present in the dataset, which can lead to unreliable value estimation. To address this issue, prior work employs techniques such as importance sampling Precup et al., 2000, regularizationKumar et al., 2020, and policy constraints Fujimoto et al., 2019. Although the method in this paper incorporates offline training, it is not pure Offline RL because the learned policy $\pi_{\mathcal{D}}$ is subsequently deployed to acquire more data and iteratively improve the reward model for further offline training. Despite this, it is worth to note that regularization, an Offline RL technique, formed a primary inspiration for this approach.

2.3 Model-Based Reinforcement Learning

Model-Based Reinforcement Learning Luo et al., 2024 is the process of learning an environment dynamics or transition function and leveraging this model for policy optimization. The learned transition model allows policies to be simulated by predicting the resulting state of actions, reducing the need for costly real-world interactions. This leads to improved sample efficiency, especially in settings where data collection is expensive or time consuming. The main problem with Model-Based RL is model accuracy and the uncertainty induced by model errors. Several approaches address this issue through uncertainty-aware modeling, such as ensemble methods. For example, Matsushima et al., 2020 trains an ensemble of K dynamics models and chooses one randomly for rollouts, while Yu et al., 2020 learns N reward models and estimates the reward as the ensemble mean penalized by the maximum standard deviation.

In addition to this, many Dyna-style Sutton, 1991 methods implement iterative model improvement to continuously refine their model. Both Feinberg et al., 2018 and Nagabandi et al., 2018 follow this paradigm by alternating between model learning and data collection with the optimized policy to progressively improve performance. Although the proposed approach is related to Model-Based RL, it does not explicitly model a transition function. Instead, only the reward model is learned and used to optimize the policy. Despite this distinction, the method adopts ideas from Model-Based RL, including ensemble networks and iterative model improvement.

Chapter 3

Preliminaries

3.1 Contextual Bandit Framework

The method in this paper was designed to be applied to a one-step, no horizon task, which is described in the Experiments 5 section. Because of this, we opt to utilize the contextual bandit Langford and Zhang, 2007 framework rather than the traditional Markov Decision Process Bellman, 1957 framework. The two frameworks are very similar, but in the contextual bandit setting, there are no state transitions. Because of this, there is no transition function and only the immediate expected reward is considered rather than the sum of expected rewards.

The contextual bandit framework is formally defined by the following: there are a number of rounds t from $t = 1, 2, 3, ..., T$. For the set of possible contexts $\mathcal{X}$ there is a context vector $x_t \in \mathcal{X}$ to describe the current trial's context or state. There is also a set of arms or actions $\mathcal{A}(x_t)$, and the agent must choose one action $a_t \in \mathcal{A}(x_t)$ during each round. After executing the action, a reward r_t is observed and determined by the reward function $r(x, a)$. In order to choose actions for each individual context, the agent implements a policy $\pi(a|x)$ or mapping from context to action. With a deterministic policy, each context results in a single action $a = \pi(x)$ with a probability of 1. For a deterministic policy represented by parameters ϕ, the goal is to find a policy that maximizes the objective or expected reward

$$J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)}\big[r\big(x, \pi_\phi(x)\big)\big]. \tag{3.1}$$

By repeatedly taking the gradient of the objective $\nabla_\phi J(\phi)$, the policy parameters can be continuously updated and push the policy towards optimality. The deterministic policy gradient from Silver et al., 2014 can then be defined by the following

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)}\big[\nabla_\phi \pi_\phi(x) \cdot \nabla_a r(x, a)|_{a=\pi_\phi(x)}\big]. \tag{3.2}$$

Please note that the original deterministic policy gradient is applied to actor-critic RL algorithms, so in the above, the the Q-function $Q^\pi(s, a)$ has been replaced with the reward function. The Q-function represents the value for taking action a in state s and following the policy π thereafter. The thing that makes the Q-function different from the reward function is that the Q-function captures the total discounted sum of future rewards while the reward function represents only the immediate reward. Because this paper deals with the formulation of contextual bandits rather than MDP's, there are no transition and resulting states to worry about, so the replacement of the Q-function with the reward function in Equations (3.1) and (3.2) is justified.

The deterministic policy gradient can be estimated from executing the policy π_ϕ in the real-world and adding the observed data to a dataset $\mathcal{D}^{\pi_\phi} = \{(x_i, a_i, r_i)\}_i^{\|\mathcal{D}^{\pi_\phi}\|}$. The dataset $\mathcal{D}^{\pi_\phi}$ can be used to estimate the deterministic policy gradient with the following

$$\nabla_\phi J(\phi) = \frac{1}{\|\mathcal{D}^{\pi_\phi}\|} \sum_{b=1}^{\|\mathcal{D}^{\pi_\phi}\|} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a r(x_b, a_b)|_{a_b=\pi_\phi(x_i)}. \tag{3.3}$$

3.2 Reward Function Modeling

As mentioned above, the observed reward r_t obtained after taking action a_t in context x_t is determined by the reward function $r(x, a)$. In many practical settings, however, the true reward function is unknown to the agent and must be inferred through interactions with the environment. One approach

is to directly learn a policy from experience by utilizing importance sampling, but an alternative is to learn an approximate reward function $\hat{r}(x, a)$ and use this for policy optimization. For any exploration policy π_{Explore}, the agent observes tuples of the form (x_t, a_t, r_t) where x_t is the context, a_t is the policy action $a_t \sim \pi_{\text{Explore}}(\cdot|x_t)$, and r_t is the resulting reward. These interactions can be used to generate a dataset $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b=1}^{\|\mathcal{D}\|}$. After populating the dataset, modeling the approximate reward function becomes a supervised learning problem.

For an approximate reward function $\hat{r_\theta}(x, a)$ with parameters θ, the goal is predict the true reward of each context-action pair. To train this model, the aim is to minimize a loss function $\mathcal{L}(y, \hat{y})$ which quantifies the discrepancy between true labels y of input x and predicted labels $\hat{y}$ from a function $f_\theta(x)$ **19**. In the reward model setting, the loss function would be defined as $\mathcal{L}(r_b, \hat{r}_\theta(x_b, a_b))$, which measures how well the predicted reward matches the true observed reward. The optimal parameters θ are obtained by minimizing the loss over a mini-batch of size B sampled from the dataset $\mathcal{D}$:

$$\theta = \arg\min_\theta \frac{1}{B} \sum_{b=1}^{B} \mathcal{L}(r_b, \hat{r}_\theta(x_b, a_b)). \tag{3.4}$$

This gradient-based optimization updates the parameters θ so that the learned reward function $\hat{r}_\theta(x, a)$ accurately approximates the true reward. Once trained, the approximate reward function can be used for policy optimization, enabling sample-efficient learning without the true reward function.

Chapter 4

Methodolgy

This section details the paper's approach to learn an effective policy. The method is organized into two distinct parts: Approximation of an Accurate Reward Model 4.1 and Iterative Policy Refinement 4.2.

4.1 Approximating and Accurate Reward Model

Before any policy learning takes place, a reward model must first be obtained. As detailed in Section 3.2, learning a reward model consists of utilizing a dataset $\mathcal{D}$ defined as $\mathcal{D} = \{(x_b, a_b, r_b)\}_{b=1}^{\|\mathcal{D}\|}$. The reward serves as the output label, and supervised learning is performed on the state-action inputs. Although this basic process can produce effective results, additional strategies are employed to further improve the accuracy and robustness of the approximate reward model. Implementing an effective data collection strategy and utilizing ensemble networks are the two primary strategies employed in this work. Together, these components help the learned reward model to produce more accurate and reliable reward estimates for state-action pairs. This accuracy is crucial, as the reward model is subsequently used to guide and optimize the policy.

4.1.1 Initial Data Collection Strategy

It is widely accepted that a model is only as good as the data on which it is trained. Ideally, a model is trained on a large and diverse dataset, enabling it to generalize across a wide range of states and actions. However, producing large-scale datasets is often time-consuming and expensive; therefore, it is essential to maximize diversity when working with limited data. To achieve this, the approach creates an initial dataset $\mathcal{D}_0$ with data generated by a human expert π_{Expert} and a random exploration policy π_{Random}. The term "expert" is used loosely here; while the actions should produce high rewards, they need not be strictly optimal. The purpose of the expert is to supply state-action pairs with relatively high rewards in order to provide a positive learning signal for the optimization procedure. In contrast, the random policy π_{Random} is likely to produce lower-reward actions, but its primary purpose is to provide a broad coverage of the state and action spaces. By combining these two data sources, the constructed dataset achieves both reward diversity and state-action coverage, improving the quality of the learned reward function.

4.1.2 Leveraging Ensemble Networks

A common strategy in supervised learning is to train an ensemble of networks rather than relying on a single model. This allows multiple predictions to be produced and aggregated in ways that improve robustness and provide estimates of uncertainty. In this work, N approximate reward models $\hat{r}_{\theta_1}, .., \hat{r}_{\theta_N}$ are trained independently with different initial parameters θ_i. The ensemble outputs are used to compute the mean predicted reward μ_θ and the standard deviation σ_θ, defined as

$$\mu_\theta(x, a) = \frac{1}{N} \sum_{i=1}^{N} \hat{r}_{\theta_i}(x, a), \tag{4.1}$$

$$\sigma_\theta(x, a) = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(\hat{r}_{\theta_i}(x, a) - \hat{\mu}_\theta(x, a)\right)^2}. \tag{4.2}$$

The predicted mean reward μ_θ serves as the primary estimate of the approximated reward. The standard deviation σ_θ is used quantify model uncertainty. Since independently trained models are likely to produce different predictions for state-action pairs that are poorly represented in the dataset, the standard deviation σ_θ provides a natural measure of uncertainty. Using these quantities, the uncertainty-penalized reward $\tilde{r}_\theta$ is defined as

$$\tilde{r}_\theta(x, a) = \mu_\theta(x, a) - \beta \cdot \sigma_\theta(x, a), \tag{4.3}$$

where the parameter β controls the degree to which uncertainty is penalized. Larger values of β will lead to more conservative reward approximations and will discourage the policy from selecting actions where the reward model is uncertain.

Algorithm 1 Train Reward Model Ensemble

Inputs: Dataset $\mathcal{D}$, number of reward networks N
Output: Reward model $\tilde{r}_\theta(x, a)$

1: Initialize reward networks $\hat{r}_{\theta_1}, ..., \hat{r}_{\theta_N}$ with random parameters $\theta_1, ..., \theta_N$
2: **for** $t = 1$ to T **do**
3: For each network $\hat{r}_{\theta_n}$, sample an independent mini-batch B_n from dataset $\mathcal{D}$
4: Update $\theta_n \leftarrow \arg\min_{\theta_n} \frac{1}{B_n} \sum_{b=1}^{B_n} \mathcal{L}(r_b, \hat{r}_{\theta_n}(x_b, a_b))$
5: **end for**
6: **return** $\tilde{r}_\theta(x, a)$

4.2 Iterative Policy Refinement

Although it is common to compute a policy by directly maximizing the reward model over a set of candidate actions, this approach becomes significantly more challenging in continuous action spaces. As a result, policy optimization is adopted as the primary strategy. After training the approximate reward model, policy optimization can be carried out using the policy gradient defined in Equation 3.2, which has been shown to be an effective approach for learning policies. However, this work employs a modified learn-

ing objective together with an iterative policy learning process to achieve improved policy performance and greater robustness compared to standard optimization.

4.2.1 Maximum Action Noise Threshold Objective

The main contribution of this paper is a modified learning objective. In reinforcement learning, the main objective is to maximize expected reward, formalized in Equation 3.1. However, when optimizing this objective in the offline setting with an approximate reward model, two issues can arise. First, the policy may exploit inaccuracies in the reward model, selecting actions that are highly rated by the approximate reward function but are actually suboptimal in the real-world. Secondly, the policy may converge to local optima due to limited exploration and the absence of online feedback. To address these challenges, we introduce a new learning objective based on a function $\eta(x, a)$, called the **Maximum Action Noise Threshold (MaxANT)** function, defined as

$$\eta(x, a) = \eta^{+}(x, a) + \eta^{-}(x, a), \tag{4.4}$$

where

$$\begin{aligned} \eta^{+}(x, a) = \max_{\boldsymbol{\Delta}} \|\boldsymbol{\Delta}\|_1 \\ \text{such that} \quad \forall \boldsymbol{\delta} \in [0, \boldsymbol{\Delta}],\ r(x, a + \boldsymbol{\delta}) \geq \tau, \end{aligned} \tag{4.5}$$

$$\begin{aligned} \eta^{-}(x, a) = \max_{\boldsymbol{\Delta}} \|\boldsymbol{\Delta}\|_1 \\ \text{such that} \quad \forall \boldsymbol{\delta} \in [0, \boldsymbol{\Delta}],\ r(x, a - \boldsymbol{\delta}) \geq \tau. \end{aligned} \tag{4.6}$$

Intuitively, $\eta(x, a)$ measures the maximum positive and negative perturbation Δ that can be applied to action a without the reward falling below a threshold τ. This function is incorporated into the modified objective

$$J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)} \big[r\big(x, \pi_\phi(x)\big) + \alpha \cdot \eta\big(x, \pi_\phi(x)\big) \big], \tag{4.7}$$

where the policy π_ϕ now aims to maximize not only expected reward, but also robustness to action perturbations. This encourages the selection of actions that not only have high reward but also lie in regions where nearby actions are similarly rated. As a result, the learned policy π_ϕ is less likely to exploit narrow peaks in the reward landscape and is more likely to converge to broad, high reward regions. This mitigates the risk of both reward exploitation and convergence to local optima. The policy gradient $\nabla_\phi J(\phi)$ for this objective is

$$\nabla_\phi J(\phi) = \mathbb{E}_{x \sim p_x(\cdot)}[\nabla_\phi \pi_\phi(x) \cdot \nabla_a (r(x,a) + \alpha \cdot \eta(x,a)|_{a=\pi_\phi(x)}], \tag{4.8}$$

which can be approximated by

$$\nabla_\phi J(\phi) = \frac{1}{B} \sum_{b=1}^{B} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a \big(r(x_b, a_b) + \alpha \cdot \eta(x_i, a_i)\big)|_{a_i = \pi_\phi(x_i)}. \tag{4.9}$$

4.2.2 Iterative Deployment of Trained Policy

After training the policy with the modified objective described above, the learned policy π_ϕ is deployed in the real world to generate a dataset $\mathcal{D}_{\pi_\phi}$. Since the learned policy $\mathcal{D}_{\pi_\phi}$ is deterministic, Gaussian noise is added to the continuous actions during its deployment to evaluate its robustness. Once collected, $\mathcal{D}_{\pi_\phi}$ is merged with the existing dataset to form $\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_\phi}$. The resulting dataset now contains a mixture of random exploration, expert, and on-policy data, increasing overall diversity. The new dataset is subsequently used to retrain the reward model, which in turn is used to further optimize the policy. By augmenting the dataset with on-policy samples, the reward model is refined in regions of the state-action space that are most relevant to the current policy. This helps correct suboptimal policy convergence that may arise from optimizing with an imperfect reward model though evaluation with real world interactions. The full algorithm is presented in Algorithm 2.

Algorithm 2 Iterative Policy Improvement with MaxANT RL

Inputs: Number of reward networks N, action noise threshold τ, action noise threshold rate α

Output: Policy π_ϕ

Initialize policy network π_ϕ with random parameters ϕ
Fill initial dataset $\mathcal{D}_0$ with data from random exploration policy π_{Random} and from expert policy π_{Expert}
for $i = 0$ to I **do**
    $\tilde{r}_\theta(x, a) \leftarrow$ TrainRewardModel$(\mathcal{D}_i, N)$
    for $t = 1$ to T **do**
        Sample mini-batch of size B from dataset $\mathcal{D}_i$
        $\nabla_\phi J(\phi) = \frac{1}{B} \sum_{b=1}^{B} \nabla_\phi \pi_\phi(x_b) \cdot \nabla_a \big(\tilde{r}(x_b, a_b) + \alpha \cdot \eta(x_b, a_b)\big)|_{a_i = \pi_\phi(x_b)}$
        Update ϕ using Equation 4.9
    end for
    Execute π_{ϕ_i} and add interactions to $\mathcal{D}_{\pi_{\phi_i}}$
    Update dataset $\mathcal{D}_{i+1} = \mathcal{D}_i \cup \mathcal{D}_{\pi_{\phi_i}}$
end for
return π_ϕ

Chapter 5

Experiments

To evaluate the proposed method, experiments were done on a sorting task with a robotic manipulator. The following experiments are the main contributions of this paper.

5.1 Task Description

The proposed task is to autonomously sort recyclable items within a constrained workspace. The item set consists of aluminum cans and plastic bottles with varying degrees of deformation, which introduces variability in shape and grasp difficulty. The manipulation is performed with a KUKA LBR iisy 3 r760 collaborative robot fitted with an Ubiros Gentle Pro gripper at the end effector. The workspace state is captured using an overhead Oak-D S2 RGBD camera and subsequently encoded to form the input to the learned policy, which outputs the corresponding control action for the manipulator. After executing the action, the robot automatically closes the gripper, moves to the recycling bin, and opens the gripper to release any grasped object. An action is considered successful if the assigned object is grasped and transported to the recycling bin. The overall sorting procedure is summarized in Algorithm 3.

Algorithm 3 Sorting

Input: Learned policy π_ϕ

```
while items in workspace do
    Camera obtains state s and encodes it as s_φ
    Robot moves to "Home" position
    Get policy action a ← π_φ(s_φ)
    Robot executes the policy action
    Robot closes gripper to grab item
    Robot moves to "Bin" position
    Robot opens gripper to drop item
end while
```

5.2 Context/State Estimation

The workspace state is obtained from an overhead RGBD camera. When the current state is requested, the camera captures an RGB and a corresponding depth image of the workspace. The RGB image is passed though a YOLO v11 instance segmentation model to acquire pixle-wise masks of bottles and cans present in the scene. The YOLO model was trained on a combination of publicly available datasets containing annotated images of general cans and bottles, along with custom images collected from the workspace with manual annotation. After obtaining the segmented instances, Principal Component Analysis (PCA) **21** is performed on each mask to compute the centroid and principal axis of the objects. The corresponding depth image is then used to recover the 3D position of the centroid. Using camera intrinsics, ray projection, and ROS2 coordinate frame transformations, the centroid is transformed into the robot's frame. The principal axis extracted from PCA defines the object's yaw angle in the robot frame. To estimate the pitch angle, a two-point method is employed. Two points located near opposite ends of the object along the principal axis are strategically chosen. Like the centroid, the 3D position of these points are computed in the robot frame using the depth image. Let the vertical change between the two points be Δz, and let the planar displacement be in the xy-plane be Δxy. The pitch angle is then computed by $\vartheta = \arctan \frac{\Delta z}{\Delta xy}$.

Due to the axial symmetry of bottles and cans, the roll angle is assumed to be zero. Therefore, the full 6D pose of each object is defined by the 3D position of the centroid and the estimated pitch and yaw angles.

5.3 Context/State Definition

Because the number of items in the workspace varies and the robot grasps one item at a time, only the 6D pose of a single target is considered in the state representation. The state captures the 3D position and orientation of the current object selected for grasping, along with contextual information about the workspace. The state is defined as:

$$s = [x, y, z, \vartheta, \psi, t, n]$$

where $x, y, z \in \mathbb{R}$ represent the Eucledian coordinates of the object's centroid, ϑ and ψ denote the pitch and yaw angles desribing the object's orientation, t represents the object type (e.g., can or bottle), and n denotes the number of objects currently present in the scene. Thus, the state respresents the 6D pose of the active object and basic scene-level information. Due to the physical constraints of the workspace, the following bounds are imposed on the state variables:

$$x \in [0.20, 0.55],\ y \in [-0.35, 0.35],\ z \in [0.80, 0.95],$$
$$\vartheta \in [-\frac{\pi}{2}, 0],\ \psi \in [-\pi, \pi]$$

Because the true state space is continuous, there are infinitely many possible states. To significantly reduce the size of the state space for the reward and policy networks, a discretized encoding is introduced. First, the workspace is divided into two equal partitions along the x and y axes, producing four spatial quadrants. Then, the pitch angle ϑ range is divided into three uniform bins, and the yaw angle ψ range is divided into four uniform bins. The object type t is preserved, while the number of objects n is removed from the discretized representation. The resulting encoded state is defined as:

$$\hat{s} = [q, w, v, t],$$

where $q \in \{1, 2, 3, 4\}$ denotes the spatial quadrant of the object, $w \in \{1, 2, 3\}$ represents the discretized pitch bin, $v \in \{1, 2, 3, 4\}$ denotes the discretized yaw bin, and t represents the object type. This encoding reduces the continuous state to a compact representation suitable for reward modeling and policy learning.

5.4 Action Definition

The actions for the sorting task define how the robotic manipulator moves prior to attempting a grasp. Each action specifies a desired relative displacement of the tool center point (TCP), i.e., the center of the gripper, with respect to the current object pose. To execute an action, the robot computes an inverse kinematics **22** solution and plans a motion to the specified pose. The action is 6-dimensional, continuous, and is defined by the following

$$a = [\Delta l, \Delta w, \Delta h, \Delta\varphi, \Delta\vartheta, \Delta\psi]$$

where $\Delta l, \Delta w, \Delta h$ are the relative Cartesian offsets in meters along the object's local XYZ frame, and $\Delta\varphi, \Delta\vartheta, \Delta\psi$ represent the relative orientation offsets (roll, pitch, and yaw, respectively) in radians. All position and orientation components are defined relative to the pose of the target object. This means that the action does not command an absolute pose in the workspace frame, but rather a pose expressed in the object's local frame. For example, the action $[0.0, 0.0, 0.0, \frac{\pi}{4}, 0.0, 0.0]$ moves the TCP to the object's centroid with matching pitch and yaw, while introducing a relative roll offset of $45°$. Each component of the relative action is clamped to pre-defined bounds defined by

$$\Delta l \in [-0.10, 0.10],\ \Delta w \in [-0.10, 0.10],\ \Delta h \in [-0.01, 0.10],$$
$$\Delta\varphi \in [-\frac{\pi}{4}, \frac{\pi}{4}],\ \Delta\vartheta \in [-\frac{\pi}{4}, \frac{\pi}{4}],\ \Delta\psi \in [-\frac{\pi}{4}, \frac{\pi}{4}]$$

to ensure the manipulator only attempts feasible grasps.

Although the policy outputs a relative action, execution requires an absolute pose in the workspace frame. The absolute action is computed with component-wise addition between the action and the true state defined in the previous section. Before execution, the resulting absolute target pose is clamped to the same bounds defined for the workspace state.

5.5 Reward Definition

Rewards are assigned autonomously to each action executed in the workspace. The reward is binary: actions that result in a successful grasp receive a reward of 1, while unsuccessful actions receive a reward of 0. Action success is determined using two criteria: a reduction in the total workspace items, and the measured current load of the gripper. If the number of items in the workspace decreases by one following the action and gripper current exceeds 150 mA, the action is classified as successful and assigned a reward of 1. Otherwise, the action is classified as a failure and assigned a reward of 0. This dual-condition check ensures that grasp attempts that fail to grab the object are not incorrectly labeled as successful.

To train the reward model, Binary Cross-Entropy (BCE) loss is used. Cross-Entropy loss **20** is widely used for probability models and classification tasks. Because the reward data takes on one of two labels (0 or 1), the binary variant of Cross-Entropy is appropriate. The Binary Cross-Entropy is defined as

$$\mathcal{L}_{BCE}(\theta) = -\frac{1}{B}\sum_{b=1}^{B} y_b \cdot \log(\hat{y}_b) + (1 - y_b) \cdot \log(1 - \hat{y}_b)$$

where y denotes the true labels of input x and $\hat{y}$ represents the predicted labels produced by a function $f_\theta(x)$. Minimizing this loss encourages the reward model to assign high probability to successful actions and low probability to unsuccessful ones.

A binary reward setup is effective for evaluating the method because multiple actions can yield successful grasps and achieve the maximum reward. However, many of these actions are suboptimal due to inherent uncertainty in the workspace and the grasping process. Rather than optimizing for any action

that simply achieves success, the modified objective encourages convergence towards actions that are more optimal and robust to uncertainty. These optimal actions are often located near the center of regions with consistently high rewards. Converging to these regions improves overall performance and helps avoid policies that settle in local optima or rely on highly situational actions.

5.6 Initial Dataset

As mentioned earlier, the goal of the initial data collection phase is to produce a dataset with diverse states, actions, and rewards. To achieve this, data was collected from both an expert and a random policy to ensure a mix of successful and unsuccessful interactions. Expert data was generated by hand-guiding the KUKA collaborative robot to a variety of feasible grasp poses for the assigned object. With this teaching process, 160 expert demonstrations were produced, corresponding to five demonstrations per state. Additionally, a random policy was implemented, sampling actions uniformly within the action bounds. Executing this policy yielded 340 additional interactions with mainly unsuccessful attempts to provide negative reward examples. Together, the expert and random data form an initial dataset of 500 interactions, containing both high quality grasps and exploration failures to produce an effective initial reward model.

Chapter 6

Conclusion

Appendix A

Bibliography

Asif, U., Tang, J., & Harrer, S. (2019). Densely supervised grasp detector (dsgd). *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*(01), 8085–8093.

Bellman, R. (1957). A markovian decision process. *Journal of mathematics and mechanics*, *6*(5), 679–684.

Detry, R., Kraft, D., Kroemer, O., Bodenhagen, L., Peters, J., Krüger, N., & Piater, J. (2011). Learning grasp affordance densities. *Paladyn*, *2*(1), 1–17.

Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., & Levine, S. (2018). Model-based value expansion for efficient model-free reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*.

Fujimoto, S., Meger, D., & Precup, D. (2019). Off-policy deep reinforcement learning without exploration. *International conference on machine learning*, 2052–2062.

Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). Reinforcement learning with deep energy-based policies. *International conference on machine learning*, 1352–1361.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International conference on machine learning*, 1861–1870.

Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023). A survey on deep reinforcement learning algorithms for robotic manipulation. *Sensors*, *23*(7), 3762.

Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems*, *33*, 1179–1191.

Langford, J., & Zhang, T. (2007). The epoch-greedy algorithm for multi-armed bandits with side information. *Advances in neural information processing systems*, *20*.

Levine, S., Pastor, P., Krizhevsky, A., Ibarz, J., & Quillen, D. (2018). Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International journal of robotics research*, *37*(4-5), 421–436.

Luo, F.-M., Xu, T., Lai, H., Chen, X.-H., Zhang, W., & Yu, Y. (2024). A survey on model-based reinforcement learning. *Science China Information Sciences*, *67*(2), 121101.

Mahler, J., Liang, J., Niyaz, S., Laskey, M., Doan, R., Liu, X., Ojea, J. A., & Goldberg, K. (2017). Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. *arXiv preprint arXiv:1703.09312*.

Matsushima, T., Furuta, H., Matsuo, Y., Nachum, O., & Gu, S. (2020). Deployment-efficient reinforcement learning via model-based offline optimization. *arXiv preprint arXiv:2006.03647*.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *nature*, *518*(7540), 529–533.

Nagabandi, A., Kahn, G., Fearing, R. S., & Levine, S. (2018). Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *2018 IEEE international conference on robotics and automation (ICRA)*, 7559–7566.

Pinto, L., & Gupta, A. (2016). Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. *2016 IEEE international conference on robotics and automation (ICRA)*, 3406–3413.

Precup, D., Sutton, R. S., & Singh, S. (2000). Eligibility traces for off-policy policy evaluation.

Prudencio, R. F., Maximo, M. R., & Colombini, E. L. (2023). A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE transactions on neural networks and learning systems*, *35*(8), 10237–10257.

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *International conference on machine learning*, 387–395.

Smith, L., Kew, J. C., Peng, X. B., Ha, S., Tan, J., & Levine, S. (2022). Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. *2022 international conference on robotics and automation (ICRA)*, 1593–1599.

Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, *2*(4), 160–163.

Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., Heess, N., Rothörl, T., Lampe, T., & Riedmiller, M. (2017). Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*.

Wu, P., Escontrela, A., Hafner, D., Abbeel, P., & Goldberg, K. (2023). Daydreamer: World models for physical robot learning. *Conference on robot learning*, 2226–2240.

Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J. Y., Levine, S., Finn, C., & Ma, T. (2020). Mopo: Model-based offline policy optimization. *Advances in neural information processing systems*, *33*, 14129–14142.